

Módulo 02 — Além do chat: System One e o Jev

Texto ou decisão? Decisões tipadas com um LLM comum, de onde vem a confiança e como medir se ela presta. Com leitura crítica do anúncio do Jev.

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M2
5. Para revisar

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. dizer, para um problema concreto, se ele pede **texto gerado** ou só uma **decisão** (classificar, rotear, pontuar, extrair, aprovar);
2. contrastar um LLM gerativo com um modelo de decisão estilo System One: **strings vs. valores tipados, um token por vez vs. tudo de uma vez, confiança implícita vs. explícita**;
3. obter uma decisão tipada de um LLM comum com Zod e explicar o que a saída restrita a um schema faz e o que ela **não** faz;
4. obter um sinal de confiança por três caminhos (logprobs, autoavaliação, autoconsistência) e **medir** se ele acompanha o acerto (calibração);
5. ler com olhar crítico o anúncio de um produto de IA: separar o que é afirmação do fornecedor, o que é medido e quais ressalvas os próprios autores fazem.

Mensagem central

Muita "IA" em software não precisa gerar texto: precisa **tomar uma decisão que o código consiga usar**. Uma decisão útil tem **tipo** (o programa não precisa adivinhar o formato) e **confiança** (o programa sabe quando chamar um humano). E confiança só vale se for **medida**.

Conceitos

O Módulo 1 mostrou que um LLM gera texto sorteando um token por vez. Este módulo pergunta: **e quando a gente não quer texto, e sim uma decisão?** Ele aprofunda a aula e cobre o que ficou de fora: como a saída restrita a um schema funciona por dentro, de onde vem um sinal de confiança e como medir se ele presta (calibração).

1. Texto ou decisão?

Grande parte da IA dentro de software não termina em um humano lendo um parágrafo. Termina num `if`:

- este ticket vai para qual fila?
- esta notícia entra na newsletter?
- este comentário viola a política?
- este documento menciona uma data de vencimento? qual?
- esta resposta do chatbot pode ser enviada ou precisa de revisão?

Nesses casos, o que o código precisa é de um **valor de um conjunto fechado** (uma categoria, um sim/não, uma nota de 1 a 5) e, idealmente, de **quão certo** o modelo está. Gerar um parágrafo e depois tentar extrair a decisão dele com regex é caro, lento e frágil, como você viu na etapa M1.

Regra de bolso:

A saída é...	Exemplo	O que usar
texto para uma pessoa ler	resumo, e-mail, resposta de chat	LLM gerativo
um valor de um conjunto fechado que o código vai usar	categoria, sim/não, nota	decisão tipada (LLM com saída estruturada, ou um modelo de decisão)
as duas coisas	classificar e justificar para o editor	decisão tipada + texto, em campos separados

2. Sistema 1 e Sistema 2

O nome "System One" vem do livro *Rápido e Devagar* (*Thinking, Fast and Slow*, 2011), de Daniel Kahneman. O livro descreve dois "modos" do pensamento humano:

- **Sistema 1:** rápido, automático, intuitivo. Reconhecer um rosto, perceber que alguém está bravo, completar "dois mais dois é...".
- **Sistema 2:** lento, deliberado, trabalhoso. Fazer uma conta de cabeça, comparar dois planos de saúde.

A analogia: muitas decisões dentro de software são "de Sistema 1": olhar um texto e julgar rapidamente. Não precisam de um modelo que escreva um ensaio para chegar à resposta.

O limite da analogia: no livro, o Sistema 1 é justamente o que mais erra por vieses. A TypeSafe argumenta que, com o treino certo, um modelo de decisões rápidas pode ser *mais* confiável. É uma afirmação a ser testada, não um fato do livro.

3. O que é o Jev

Em 15 de setembro de 2026, a TypeSafe AI anunciou uma classe de modelos que chamou de **System One Models**, e o primeiro deles, o **Jev** ([post de lançamento](#)). Segundo o post:

- **Não gera texto.** Recebe um **estado** não estruturado (um texto, um JSON, uma conversa) e **perguntas tipadas**, e devolve **respostas tipadas com probabilidades**.
- **Gera em paralelo.** Todas as saídas saem "em uma única consulta", não um token de cada vez.
- **Comunica confiança** em toda resposta, e o post afirma que ela é **calibrada**: confiança maior significa acurácia maior.
- **É treinado com RLCD** (*Reinforcement Learning for Calibrated Decisions*), em vez de RLHF/RLVR, otimizando "respostas com probabilidades epistemicamente honestas".
- **Uso pretendido:** um "if inteligente" dentro do código: classificar, rotear, pontuar, extrair, servir de guardrail ou de juiz.

A documentação define três tipos de pergunta ([primitivas](#)):

Primitiva	Pergunta	Resposta
Choice	escolha uma opção de uma lista	a opção, a probabilidade de cada uma e a confiança
Score	dê uma nota em níveis ordenados	a nota (pode cair entre níveis), a probabilidade de cada nível e a confiança
Noul	sim ou não?	a probabilidade de "sim", de 0 a 1

Strings x tipos, sequencial x paralelo

Comparando com o Módulo 1:

	LLM gerativo	System One (segundo o fornecedor)
Saída	texto (string)	valores tipados
Como gera	um token por vez, cada um condicionado nos anteriores	tudo de uma vez
Erro de formato	possível (o texto pode vir em qualquer formato)	impossível por construção (só existem as opções definidas)
Confiança	implícita (nos logprobs, se o provedor expõe) ou pedida em texto	explícita em toda resposta
Bom para	texto para humanos, raciocínio longo, tarefas abertas	decisões rápidas e repetidas dentro de sistemas

Confiança não é probabilidade

A documentação separa os dois conceitos ([confiança](#)). Numa Choice, a confiança é derivada da distribuição de probabilidades:

$$\text{confiança} = (n \times p_{\text{max}} - 1) / (n - 1)$$

probabilidade n = número de opções, p_{max} = maior

Com 3 opções e 88% na favorita, a confiança é $(3 \times 0,88 - 1) / 2 = 0,82$. Com 2 opções e 50% em cada, é 0 (o modelo não decidiu nada). A ideia: 40% numa escolha entre 10 opções é bem mais "decidido" do que 40% numa escolha entre 2. Você implementa essa fórmula no exercício.

4. Lendo o anúncio com olhar crítico

O post traz números fortes: latência de 70 a 500 ms, de 40x a 200x mais rápido que modelos de fronteira, entrada a US\$ 0,042 por milhão de tokens e saída "grátis", e "0%" de erro de tipo. **São números do próprio fornecedor**, o produto está em **early access**, e os autores listam ressalvas. As principais, traduzidas do post:

- a consulta de demonstração "é muito simplificada", e as perguntas foram escolhidas com nomes legíveis para a tela ficar compreensível;
- a entrada relativamente curta "favorece o nosso modelo";
- os ganhos nos workflows avaliados devem estar "no topo do que se vê no mundo real";
- esses workflows foram montados por pessoas do time do próprio produto, então "pode haver viés";
- a resposta de referência foi a média de dois modelos de outras empresas, o que puxa a avaliação na direção deles;

- os LLMs comparados passaram por um "wrapper" da própria TypeSafe que força saída estruturada com probabilidades, o que os deixa mais lentos e caros do que seriam sem isso;
- os números dos LLMs vieram de um agregador de APIs, e "quase certamente há viés aí";
- o "0% de erro de tipo" não é medido: é garantido pela construção (o modelo só consegue devolver opções válidas).

A própria documentação tem uma página de limitações conhecidas do `jv-1.13` (*jaggedness*): leitura muito literal das instruções, contas e contagens, comparação de datas, perguntas com várias "indireções", estados grandes cheios de detalhe irrelevante, conteúdo adversarial, ordem das opções numa Choice e geração de texto ("use um modelo gerativo"). A recomendação deles é manter contas e datas no código e dar ao modelo só o julgamento.

Checklist para qualquer anúncio de IA (inclusive este):

1. Quem mediu? O fornecedor, um terceiro, ou você?
2. Em que tarefa, com que dados, comparado com quê e configurado como?
3. Os autores listam ressalvas? Quais?
4. O produto está disponível para você reproduzir?
5. **O resultado se repete na sua tarefa, com os seus dados?** Essa é a única pergunta que decide.

"Erro de tipo zero" e "decisão certa" são coisas diferentes: um modelo pode devolver sempre uma categoria válida e ainda assim escolher a categoria errada.

5. Decisão tipada com um LLM comum

Não é preciso o Jev para ter decisões tipadas. A maioria dos provedores aceita **saída estruturada**: você manda um JSON Schema (no nosso caso, gerado a partir de um schema Zod) e o provedor garante que a resposta segue o formato.

Como funciona por dentro: decodificação restrita

Lembre do Módulo 1: em cada passo, o modelo dá uma probabilidade para cada token e um é sorteado. Na **decodificação restrita** (*constrained decoding*), antes do sorteio, o provedor **zera a probabilidade de todo token que quebraria o schema** e renormaliza o resto. É parecido com o top-p, mas o filtro é "o que é válido no formato" em vez de "o que é provável".

Consequências:

- **O formato é garantido** (com provedores que implementam isso de verdade).
- **O conteúdo não melhora**. O modelo continua "querendo" os mesmos tokens; o schema só proíbe alguns.
- **Pode piorar**. Se o modelo queria escrever algo fora da lista, ele é empurrado para a opção válida mais parecida no nível do **token**, não do significado.

A armadilha do rótulo

Na preparação desta aula, pedimos ao modelo `{ "rotulo": <uma de: modelos, ferramentas, pesquisa, regulacao, mercado, irrelevante> }`. No token em que ele decide, o modelo queria escrever `IA` ($\approx 70\%$) ou `re`, de "relevante" ($\approx 15\text{--}20\%$). Nenhuma opção começa com "IA". A única que começa com "re" é `regulacao`. Resultado: notícias de ferramentas e de pesquisa viraram "regulação".

Como evitar:

- separe as perguntas: primeiro `relevante: boolean`, depois `categoria`;
- use nomes de campo e de opção que digam o que significam;
- **ordem dos campos importa**: o modelo gera o JSON da esquerda para a direita, então o que vem antes influencia o que vem depois. Um campo de justificativa *antes* da decisão funciona como "pensar antes de responder";
- teste com exemplos reais e olhe as respostas, não só se o formato está certo.

Formato restrito pode atrapalhar o raciocínio

Tam et al. (2024) encontraram queda significativa de desempenho em tarefas de raciocínio quando o modelo é obrigado a responder num formato restrito, e queda maior quanto mais rígido o formato. Para decisões simples (classificar uma notícia), o efeito tende a ser pequeno; para tarefas que exigem raciocínio, considere deixar um campo de texto livre para o modelo "pensar" antes do campo da decisão.

O código no controle

Saída estruturada resolve o formato. O resto continua sendo trabalho do seu código:

- **validar** (Zod): o tipo bate? a confiança está entre 0 e 1?
- **aplicar regras determinísticas**: "se não é relevante, a categoria é nula" não precisa de IA;
- **decidir o que fazer quando falha**: saída inválida $\neq$ erro de rede;
- **guardar** o que foi decidido e com que confiança, para medir depois.

6. De onde vem um sinal de confiança

Com um LLM comum, há três caminhos.

6.1 Logprobs

Se o provedor devolve as probabilidades dos tokens, dá para olhar o token em que o modelo "decide" e ver quanto de probabilidade cada opção recebeu. Cuidados:

- as alternativas incluem tokens que não são opções (IA , **): é preciso filtrar e renormalizar (você faz isso no exercício);
- opções que começam igual (re...) ficam ambíguas no primeiro token;
- **modelos pós-treinados costumam ficar confiantes demais nos próprios tokens.** O relatório técnico do GPT-4 mostra que o modelo só pré-treinado era bem calibrado e que o pós-treino reduziu essa calibração (OpenAI, 2023). No nosso teste, o token do booleano relevante saiu com ≈100% em todas as notícias: não serviu de sinal.

6.2 Autoavaliação (confiança verbalizada)

Pedir ao modelo um campo `confiança` no JSON. É o caminho mais simples e funciona com qualquer provedor. Pesquisas mostram resultados mistos:

- Tian et al. (2023): para modelos com RLHF, a confiança escrita como texto costuma ser **mais bem calibrada** que as probabilidades internas, muitas vezes reduzindo o erro de calibração (ECE) em cerca de 50% relativos;
- Xiong et al. (2023): quando verbalizam a confiança, os LLMs **tendem a ser confiantes demais**, talvez imitando como humanos expressam certeza; modelos maiores se saem melhor;
- Kadavath et al. (2022): modelos maiores conseguem avaliar razoavelmente se as próprias respostas estão corretas, quando a pergunta é feita no formato certo.

No nosso teste com um modelo de 4B, a confiança verbalizada ficou entre 0,90 e 0,99 em **todas** as notícias, incluindo as 5 erradas.

Atenção à pergunta: numa primeira versão do prompt, o modelo devolveu `confiança: 0.15` para notícias que ele classificou corretamente como irrelevantes. Ele entendeu "confiança" como "chance de ser relevante". Diga explicitamente que a confiança é sobre a classificação inteira estar correta.

6.3 Autoconsistência

Rodar o mesmo pedido várias vezes com temperatura > 0 e contar os votos: se 5 de 5 execuções dão a mesma resposta, a confiança é 1; se 3 de 5, é 0,6. A ideia vem de Wang et al. (2022), que usaram a votação para melhorar o raciocínio. Custo: N chamadas em vez de uma. No nosso teste, a votação deu 5 de 5 também nos erros: o modelo erra **de forma consistente** quando o erro vem de uma interpretação (por exemplo, achar que um caso de empresa é "ferramentas").

7. Calibração: a confiança presta?

Um modelo é **calibrado** quando, entre todas as vezes em que ele diz 80% de confiança, ele acerta mais ou menos 80%. Calibração não é acurácia: um modelo pode acertar pouco e ser calibrado (se disser que tem pouca certeza), ou acertar muito e ser descalibrado.

Tabela (ou diagrama) de calibração

Agrupe as previsões em faixas de confiança e compare, em cada faixa, a confiança média com a acurácia:

Faixa	Previsões	Confiança média	Acurácia
0,9–1,0	20	97%	75%

Esse foi o nosso resultado: todas as previsões caíram na faixa mais alta, e a acurácia ficou 22 pontos abaixo da confiança. Desenhada num gráfico (confiança no eixo x, acurácia no y), a tabela vira um **diagrama de confiabilidade** (*reliability diagram*): um modelo calibrado fica em cima da diagonal.

Métricas

- **ECE** (*expected calibration error*): média, ponderada pelo tamanho de cada faixa, da distância entre acurácia e confiança. 0 = perfeito. Nosso resultado: $\approx 0,22$.
- **Brier score**: média de $(\text{confiança} - \text{acerto})^2$, com acerto = 1 ou 0. Mistura calibração e acurácia. 0 = perfeito.
- **Cobertura x acurácia**: "se eu só automatizar acima de um limiar, quanto eu cubro e quanto eu acerto?". No nosso teste, aceitando só $\geq 0,99$, a cobertura cai para 60% e a acurácia sobe para 92%. A confiança estava descalibrada, mas ainda **ordenava** os casos um pouco. Essa curva é a base para decidir quando chamar um humano (Módulo 7).

Com 20 exemplos, todos esses números são **muito instáveis**: uma notícia muda a acurácia em 5 pontos. Para decidir algo de verdade, você precisa de mais dados rotulados. É assunto do Módulo 3.

Recalibrar: temperature scaling

Guo et al. (2017) mostraram que redes neurais modernas tendem a ser confiantes demais e que um ajuste simples resolve boa parte do problema: dividir os logits por uma temperatura T (a mesma do Módulo 1), escolhida num conjunto de validação. $T > 1$ "achata" as probabilidades de um modelo confiante demais. Você implementa isso no desafio do exercício.

O jeito "System One" de resolver o mesmo problema é treinar o modelo para já sair calibrado (o RLCD da TypeSafe). Em qualquer caso, a pergunta final é a mesma: **medido nos seus dados, o número bate?**

Referências

- TypeSafe AI. *Introducing System One Models and Jev* (15/set/2026).
<https://typesafe.ai/blog/introducing-system-one-models-and-jev>

- TypeSafe AI, documentação: primitivas (<https://docs.typesafe.ai/primitives>), confiança (<https://docs.typesafe.ai/confidence>), API (<https://docs.typesafe.ai/api>), limitações do jev-1.13 (<https://docs.typesafe.ai/model-jaggedness/jev-1.13>)
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- Guo et al. (2017). *On Calibration of Modern Neural Networks*. <https://arxiv.org/abs/1706.04599>
- Kadavath et al. (2022). *Language Models (Mostly) Know What They Know*. <https://arxiv.org/abs/2207.05221>
- Wang et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. <https://arxiv.org/abs/2203.11171>
- OpenAI (2023). *GPT-4 Technical Report*. <https://arxiv.org/abs/2303.08774>
- Tian et al. (2023). *Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback*. <https://arxiv.org/abs/2305.14975>
- Xiong et al. (2023). *Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs*. <https://arxiv.org/abs/2306.13063>
- Tam et al. (2024). *Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models*. <https://arxiv.org/abs/2408.02442>
- AI SDK — saída estruturada: <https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data>

Exercício

Tarefa assíncrona. Os testes rodam offline: não precisa de modelo rodando.

Na aula, a pergunta foi: **a confiança que o modelo informa acompanha o acerto?** Neste exercício você implementa as ferramentas para responder isso: três jeitos de obter um sinal de confiança de um LLM comum e as métricas que dizem se esse sinal presta.

Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex02-decisoes test          # versão base
pnpm -F @mentoria/ex02-decisoes test:desafio   # desafio extra
```

Um teste só:

```
pnpm -F @mentoria/ex02-decisoes exec vitest run test/calibracao.test.ts -t "ece"
```

Os testes **falham** até você implementar. A solução de referência está em [solucao/](#). Tente antes de abrir. `pnpm -F @mentoria/ex02-decisoes test:solucao` roda os mesmos testes contra ela.

Versão base

Arquivo	Função	O que faz
src/confianca.ts	probabilidadesDasOpcoes	a partir das alternativas (logprobs) do token de decisão, a probabilidade de cada opção válida
	confiancaDaEscolha	a fórmula de confiança de uma Choice documentada pela TypeSafe: $(n \times p_{\text{max}} - 1) / (n - 1)$
	votacao	autoconsistência: a resposta mais votada e a fração de votos
src/calibracao.ts	acuracia, brier	métricas básicas
	tabelaDeCalibracao	agrupa as previsões por faixa de confiança

Arquivo	Função	O que faz
	<code>ece</code>	erro de calibração esperado
	<code>coberturaVsAcuracia</code>	se eu só aceitar acima de um limiar, quanto cubro e quanto acerto?

Sugestão de ordem: `confiancaDaEscolha` → `votacao` → `acuracia` → `brier` → `tabelaDeCalibracao` → `ece` → `coberturaVsAcuracia` → `probabilidadesDasOpcoes`.

Dicas:

- `logprob` é o **logaritmo** da probabilidade: use `Math.exp(logprob)` para voltar à probabilidade.
- Em `tabelaDeCalibracao`, cuidado com a confiança 1: ela tem que cair na última faixa, não numa faixa nova.
- `ece` fica bem curto se você reaproveitar `tabelaDeCalibracao`.

Feito = `pnpm -F @mentoria/ex02-decisoes test` com todos os testes passando.

Use nos seus dados

Depois de fazer a etapa M2 do projeto final, o comando `pnpm -F @mentoria/curador m02:comparar` grava `projeto-final/saidas/m02-previsoes.json` (uma lista de `{ id, confianca, acertou }`). Escreva um script curto que leia esse arquivo e imprima a acurácia, a confiança média, o ECE e a cobertura × acurácia para os limiares 0,9, 0,95 e 0,99 usando as suas funções. É o que a demo `demo:calibracao` da aula faz. Compare com o resultado da turma.

Desafio extra — temperature scaling

Em `src/desafio/temperatura.ts`, implemente:

- `logVerossimilhancaNegativa(exemplos, T)`: o quanto um modelo com temperatura T "explica" os rótulos (menor = melhor);
- `ajustarTemperatura(exemplos)`: a temperatura que minimiza essa métrica.

É a mesma temperatura do Módulo 1, agora usada para **calibrar**: um modelo confiante demais precisa de $T > 1$. O teste usa um modelo que dá ~96% para a classe que só está certa em 70% dos casos e espera $T \approx 2,6$. A técnica é de [Guo et al. \(2017\)](#).

Feito = `pnpm -F @mentoria/ex02-decisoes test:desafio` com todos os testes passando.

Para ir além (sem teste): por que *temperature scaling* não muda a acurácia do modelo, só a confiança?

Perguntas para pensar

1. Em `probabilidadesDasOpcoes`, por que um token que começa duas opções (`re` → `regulacao` e `relevante`) precisa ser ignorado? O que isso diz sobre escolher nomes de opções?
2. Por que a confiança de uma Choice usa o número de opções e não só a maior probabilidade?
3. A votação por autoconsistência deu 5 de 5 em respostas erradas. Que tipo de erro a autoconsistência **não** consegue detectar?
4. Com 20 previsões, quanto a acurácia muda se uma única previsão virar de errada para certa? O que isso diz sobre tirar conclusões de um ECE calculado com poucos dados?

Projeto final — etapa M2

Etapa M2 — decisão tipada com confiança

Objetivo: em vez de texto livre, o modelo devolve um objeto que o código usa direto, `{ relevante, categoria, confianca }`, validado por Zod. O código fica no controle das regras e das falhas.

Implemente em `src/m02/classificar-tipado.ts`:

1. `SchemaClassificacao`: `relevante` (boolean), `categoria` (uma de `CATEGORIAS` ou `null`), `confianca` (número de 0 a 1).
2. `montarPromptTipado(noticia) → { system, prompt }`. Explique o que é `confianca`: a certeza de que a classificação **inteira** está correta (na preparação desta etapa, sem essa explicação, o modelo devolveu 0,15 para notícias irrelevantes que ele tinha acertado: entendeu "confiança" como "chance de ser relevante").
3. `classificarTipado(noticia, { temperature, modelo }) → { ok: true, decisao } ou { ok: false, motivo }`:
 - saída fora do schema → `motivo: "saida-invalida"` (outros erros, como rede, continuam sendo lançados);
 - não relevante → `categoria` vira `null` no código;
 - relevante sem categoria → `motivo: "inconsistente"`.

Cuidado com nomes: a saída restrita a um schema só proíbe tokens. Se o começo do que o modelo quer escrever bate com o começo de uma opção, ele "cai" nela (veja a demo `demo:armadilha-do-rotulo` do Módulo 2). Por isso `relevante` vem antes e separado de `categoria`.

Feito =

1. `pnpm -F @mentoria/curador exec vitest run test/m02.test.ts verde` (offline, com mock);
2. a comparação roda com o modelo de verdade nas notícias rotuladas e grava `saidas/m02-comparacao.md` e `saidas/m02-previsoes.json`:

```
pnpm -F @mentoria/curador m02:comparar
```

Abra o relatório e responda (vamos discutir na Aula 3):

- O texto livre e a decisão tipada acertaram quanto? O que mudou além da acurácia?
- A confiança acompanha o acerto? Em que faixa estão os erros?
- Os erros são aleatórios ou têm padrão? O rótulo humano está sempre certo?

Desafio extra: acrescente um campo `justificativa` (string curta) **antes** de `relevante` no schema e rode a comparação de novo. A acurácia mudou? E a confiança? (A ordem dos campos importa: o modelo gera o JSON da esquerda para a direita.)

Para revisar

Perguntas para discussão (15 min)

1. Pense num sistema do seu dia a dia que usa (ou poderia usar) IA. A saída é texto para uma pessoa ler ou uma decisão que o código usa?
2. Se a decisão tipada acertou o mesmo tanto que o texto livre, o que ganhamos com ela?
3. O modelo disse 0,99 de confiança e errou. De quem é a culpa e o que você faria antes de usar essa confiança para automatizar algo?
4. O post do Jev diz que o modelo "nunca erra o tipo". Por que isso é verdade por construção e por que não é a mesma coisa que acertar a decisão?
5. Que evidência você pediria antes de trocar um LLM por um modelo de decisão como o Jev no seu sistema?

Armadilhas comuns

- **Achar que saída estruturada deixa o modelo mais inteligente.** Ela garante o formato, não o conteúdo.
- **Nomes de opções que colidem com o que o modelo quer escrever.** O schema só proíbe tokens; se o começo de uma palavra que o modelo quer bate com o começo de uma opção, ele "cai" nela. Use nomes descritivos e campos separados (ex.: `relevante` antes de `categoria`).
- **Tratar a confiança verbalizada como probabilidade.** O número que o modelo escreve é texto gerado, como qualquer outro. Pode ser útil, mas só depois de medido.
- **Usar logprobs de um token "saturado".** Depois do pós-treino, muitos modelos dão ~100% para o token escolhido; isso não diz nada sobre acerto.
- **Pôr regra de negócio no prompt** quando ela pode ficar no código (ex.: "se não é relevante, a categoria é nula").
- **Engolir todo erro como "saída inválida".** Erro de rede não é erro de schema; trate separado.
- **Aceitar número de fornecedor sem ler as ressalvas.**